AI-Powered Adaptive Path Planning and Collision Avoidance for Autonomous Vehicles on Unstructured Indian Roads

Smart India Hackathon (SIH) – Complete Technical Documentation & Team Handover

Prepared from the project development record and team handover notes

Contents

1. Executive Summary
2. 1. SIH Problem Statement and Motivation
3. 2. Project Objectives
4. 3. Technology Stack and MATLAB Products
5. 4. Five Scenario Validation Suite
6. 5. Initial MATLAB Scenario Development
7. 6. Visualization and Simulation-Pacing Work
8. 7. Trajectory and Starting-Position Debugging
9. 8. Collision Avoidance and Safety Architecture
10. 9. Deep Learning Pipeline
11. 10. ReLU Activation in the CNN
12. 11. Dataset Integration Strategy
13. 12. Model Training and Evaluation
14. 13. False-Positive Handling and YOLO Threshold Strategy
15. 14. Stateflow / Decision Logic
16. 15. Project File and Module Organization
17. 16. Performance Metrics
18. 17. Reported Scenario Results
19. 18. Video and Demonstration Workflow
20. 19. Major Engineering Problems Solved
21. 20. Why a Hybrid AI + Rule-Based System
22. 21. Judge Questions and Recommended Answers
23. 22. Current Development Status
24. 23. Recommended Finalization Checklist
25. 24. Team Handover Summary

1. Executive Summary

This project develops a MATLAB-based autonomous-driving simulation framework designed specifically for unstructured Indian road conditions. The system is intended to handle missing lane markings, mixed traffic, irregular vehicle and pedestrian motion, sudden obstacles, slow-moving vehicles, market congestion, and cattle crossings.

The implementation combines scenario simulation, trajectory generation, path tracking, collision-avoidance logic, deep-learning-based perception, semantic road understanding, behavioral control, telemetry, performance evaluation, and presentation-oriented visualization. Five scenarios are used as the principal validation set: an unmarked village road, a busy urban intersection, a highway merge with slow vehicles, a dense market area, and a sudden cattle-crossing event.

2. 1. SIH Problem Statement and Motivation

Conventional autonomous-driving systems are often developed around structured roads with clear lane markings, predictable traffic flow, standardized signage, and controlled intersections. Indian roads frequently contain heterogeneous road users and less predictable behavior. Cars, buses, trucks, auto-rickshaws, two-wheelers, bicycles, pedestrians, pushcarts, and animals can share the same road space.

The project therefore focuses on adaptive navigation in free space rather than relying only on lane-following. The system is designed to perceive hazards, estimate their relevance, make a driving decision, generate or adjust a safe path, and maintain vehicle motion while retaining a deterministic safety fallback.

3. 2. Project Objectives

- Build a MATLAB/Simulink-oriented autonomous-navigation simulation pipeline.
- Validate the system on all five required Indian-road scenarios.
- Model unstructured roads and irregular traffic behavior.
- Add object detection and road-segmentation capabilities using deep learning.
- Use ReLU-based CNN architecture for learned steering and speed prediction.
- Combine learned perception with deterministic collision-avoidance rules.
- Generate measurable results such as completion, clearance, braking events, and latency.
- Produce visual demonstrations and documentation suitable for SIH judging.

4. 3. Technology Stack and MATLAB Products

Core platform: MATLAB and Simulink.

Main MATLAB capabilities used/planned: Automated Driving Toolbox for driving scenarios, actors and scenario visualization; Navigation Toolbox for path-following/planning concepts; Stateflow for decision/state-machine logic; Deep Learning Toolbox for neural-network training and inference; Computer Vision functionality for object detection and image processing; Sensor Fusion and Tracking Toolbox for multi-object tracking and sensor-fusion concepts; Vehicle Dynamics Blockset or Simulink vehicle models for vehicle-behavior validation.

RoadRunner was considered for detailed 3D scenario creation, but the project also established a MATLAB-native route using Driving Scenario Designer / drivingScenario so development can continue even when RoadRunner is not available under the academic license.

5. 4. Five Scenario Validation Suite

Scenario 1 – Unmarked Village Road: A narrow, winding road without conventional lane markings. The main hazard is an unexpected pedestrian crossing, with the broader design also considering potholes and unclear road edges.

Scenario 2 – Busy Urban Intersection: A four-way intersection without signals, containing mixed traffic such as auto-rickshaws, bicycles and buses. The objective is yielding, creeping, and conflict-aware navigation.

Scenario 3 – Highway Merge with Slow Vehicles: A higher-speed environment containing slow-moving heavy vehicles such as trucks. The objective is safe merging and overtaking while respecting vehicle motion constraints.

Scenario 4 – Dense Market Area: A narrow, congested corridor with pedestrians, pushcarts, parked or moving auto-rickshaws, and roadside activity. The objective is cautious stop-and-go navigation and lateral avoidance.

Scenario 5 – Sudden Cattle Crossing: A rural scene where cattle enter the ego vehicle's path unexpectedly. The intended behavior is a state sequence of approach, detection, stopping, safe path replanning, and resumption.

6. 5. Initial MATLAB Scenario Development

The early prototype used drivingScenario, road centers, lane specifications, actors, trajectories, controllerPurePursuit, and chasePlot. The first working scenario established a narrow road, an ego vehicle, and a pedestrian with a crossing trajectory.

During development, several API and syntax issues were discovered and corrected. These included replacing an incorrect laneSpecification call with lanespec, removing an unsupported RoadMaterial argument, replacing invalid pushTrajectory usage with trajectory/smoothTrajectory, removing the invalid setup(scenario) call, replacing deprecated MaxAngularVelocity usage with MaxCurvature, and removing invalid chasePlot arguments/output assignments.

7. 6. Visualization and Simulation-Pacing Work

A major practical problem was that the simulation ran much faster than real time. The vehicle appeared to jump toward the end of the road even though the underlying trajectory was progressing. A global SIM_SLOWDOWN concept was introduced to pace each frame according to scenario sample time.

The pacing approach measures elapsed wall-clock time and delays the next frame when necessary. A slowdown factor of 1 represents approximately real-time playback, while higher values intentionally slow the demonstration so the complete journey and lateral movements are visible.

Additional visualization work included native chasePlot usage, top-view scenario visualization, traced ego paths, explicit axes for long horizontal roads, and a final-frame delay between scenarios. These changes were aimed at making the system understandable during live SIH demonstrations.

8. 7. Trajectory and Starting-Position Debugging

A significant bug occurred when smoothTrajectory was repeatedly recomputed using waypoint lists that still contained points behind the ego vehicle. This could cause apparent resets or incorrect starting positions.

The development direction was changed toward a remaining-waypoints/masking approach: only points strictly ahead of the ego vehicle are considered, and the current ego position can be used as the trajectory anchor. This avoids resetting the vehicle to the original road origin during replanning.

The documented debugging conclusion was that part of the apparent teleportation was visual: the simulator was progressing several times faster than the display. Pacing therefore became a core part of the demonstration architecture.

9. 8. Collision Avoidance and Safety Architecture

The prototype initially used direct geometric distance checks between the ego vehicle and hazards. A safety threshold of approximately 12 m was used in the prototype to trigger emergency braking when a relevant pedestrian or obstacle was sufficiently close.

The final architecture is intentionally hybrid. Deep-learning models are used for perception and classification, while a deterministic safety layer remains active as a fallback. This means a perception confidence problem does not automatically remove the emergency-braking mechanism.

The intended hierarchy is: perception → object/road understanding → prediction → decision/state machine → path generation/control → vehicle motion, with an independent safety monitor capable of overriding normal speed commands.

10. 9. Deep Learning Pipeline

Three learning components were selected for the project.

A. YOLO Object Detection: Intended for real-time detection of classes such as person, cow/cattle, auto-rickshaw, bicycle and traffic-light-related objects. It replaces or supplements simple distance-only perception with bounding-box-based visual perception.

B. DeepLabv3+ Semantic Segmentation: Intended to identify drivable-road pixels and road boundaries. This is especially relevant to unmarked village and dense-market environments where painted lane lines are unreliable or absent.

C. CNN Behavioral Cloning: Intended to map visual frames to continuous control outputs, particularly steering and speed. The model is used as a learned-control component that can be blended with deterministic planning/control rather than replacing the safety layer entirely.

11. 10. ReLU Activation in the CNN

ReLU (Rectified Linear Unit) is explicitly part of the CNN behavioral-cloning architecture. Its mathematical definition is f(x) = max(0, x). ReLU layers are placed after convolutional and hidden fully connected layers to introduce non-linearity.

The architecture described during development contains three convolutional blocks with ReLU activations, followed by max-pooling, a fully connected hidden layer with another ReLU activation, dropout, and a two-value regression output representing steering and speed.

For SIH discussion, ReLU can be explained as a computationally simple activation that helps neural networks learn non-linear visual relationships efficiently. For positive inputs, its gradient remains active, which generally supports effective gradient-based training compared with saturating activations.

12. 11. Dataset Integration Strategy

The project uses the India Driving Dataset (IDD) and JSON annotations as the main real-world visual-data source. Additional traffic-light structured datasets and testing/training data were also mentioned as available project data. Some data were transferred only partially because of network limitations, so the development workflow also considered synthetic samples for experimentation and pipeline stability.

The IDD annotation pipeline was designed to handle polygon-style annotations by converting polygons into bounding boxes and mapping dataset labels into project classes such as person, cow, auto-rickshaw and bicycle. The documented development run reported that thousands of annotation JSON files were being parsed and that available real images were supplemented with synthetic samples.

13. 12. Model Training and Evaluation

The intended training pipeline contains separate stages for data acquisition, annotation parsing, model training, inference, and evaluation. The project structure includes training scripts for YOLO, DeepLabv3+, and behavioral cloning, plus an evaluation stage for metrics such as mAP, IoU, and MAE.

Important status distinction for the team: the scenario simulator and rule-based safety components were reported as working, while the deep-learning models were being trained/integrated incrementally. Therefore, during a judge presentation, the team should state clearly which model is actually trained on the final dataset, which is fine-tuned, and which results are from fallback/synthetic experiments.

The project documentation previously reported an ML-enhanced 5/5 scenario run and specific scenario metrics. These numbers should be treated as reported development-run results and revalidated before being presented as final benchmark results.

14. 13. False-Positive Handling and YOLO Threshold Strategy

During development, a low confidence threshold caused excessive detections, especially in the market scenario. A documented example reported hundreds of false pedestrian detections, which caused speed to remain artificially capped.

The mitigation strategy separates thresholds by perception method: a stricter threshold for sliding-window CNN inference, a conservative threshold for color-based fallback, and a lower threshold where a YOLO detector is sufficiently reliable. The exact threshold should be tuned against a validation set before final SIH reporting.

The deterministic collision-avoidance layer remains the safety net. Deep learning is therefore an enhancement to perception rather than the sole authority for emergency braking.

15. 14. Stateflow / Decision Logic

The project architecture uses a finite-state-machine concept for Indian-road behavior. Important states include cruising/free-space navigation, yielding or slowing, overtaking/lateral shifting, emergency braking, and the dedicated cattle-crossing sequence.

Scenario 5 is the clearest example of the intended state machine: APPROACH → CATTLE_DETECTED → STOPPED → REPLANNING → CLEAR/RESUME. This sequence provides an intuitive explanation for judges because it connects perception, decision-making and planning into a closed loop.

16. 15. Project File and Module Organization

The documented project structure is centered around a master RunAllScenarios script, five scenario modules, machine-learning utilities, and shared safety/metrics utilities.

Scenario modules: Scenario1_VillageRoad, Scenario2_UrbanIntersection, Scenario3_HighwayMerge, Scenario4_MarketArea, Scenario5_CattleCrossing.

Shared utilities described in the work include CollisionAvoidance, MetricsLogger, LogStep, PrintSummary and PlotResults. ML components include data download, IDD annotation parsing, YOLO training/inference, DeepLab inference/training, CNN behavioral cloning, and model evaluation.

17. 16. Performance Metrics

The project tracks or proposes: scenario completion rate, collision rate, minimum clearance, average/minimum speed, braking-event count, replanning latency, path smoothness/curvature and jerk-related measures.

A documented development run reported: 5/5 scenarios passed, 100% completion, 0% recorded collisions, and approximately 0.05 s / 20 Hz frame processing in the simulation pipeline. Reported scenario values included S1 minimum clearance around 11.90 m, S2 around 4.08 m, S3 around 11.29 m, S4 around 4.02 m and S5 around 13.81 m. These should be regenerated and verified before final submission.

18. 17. Reported Scenario Results

S1 Village Road: reported speed 6.00 m/s, minimum clearance 11.90 m, 1 braking event, PASS.

S2 Urban Intersection: reported speed 6.55 m/s, minimum clearance 4.08 m, 14 braking events, PASS.

S3 Highway Merge: reported speed 18.01 m/s (about 71 km/h), minimum clearance 11.29 m, 1 braking event, PASS.

S4 Market Area: reported speed 3.14 m/s, minimum clearance 4.02 m, 11 stop-and-go/braking events, PASS.

S5 Cattle Crossing: reported speed 11.93 m/s, minimum clearance 13.81 m, 0 braking events in the recorded run, PASS.

19. 18. Video and Demonstration Workflow

The project experimented with automated video recording using MATLAB VideoWriter. Motion JPEG AVI was selected during troubleshooting because it avoided an MPEG-4/codec issue encountered in the development workflow.

The recommended demonstration flow is to run the five scenarios through the master script, show the 3D chase view and top-view behavior, display the final performance dashboard, and use recorded clips for the SIH presentation/PPT.

20. 19. Major Engineering Problems Solved

- Incorrect/unsupported MATLAB function names and properties.
- Invalid trajectory APIs and conflicting trajectory/manual-motion control.
- Blank or repeatedly cleared visualization windows.

- Deprecated Pure Pursuit properties.
- Excessive simulation speed that made motion appear instantaneous.
- Ego vehicle starting too close to the road end.
- Per-frame trajectory replanning causing apparent resets.
- YOLO/CNN false positives causing excessive speed reduction.
- Long horizontal road plots appearing as thin stripes because of automatic axis scaling.
- Scenario-to-scenario transition and figure-handle cleanup problems.

21. 20. Why a Hybrid AI + Rule-Based System

The project does not depend entirely on a neural network. Learned models provide richer perception and learned control behavior, while deterministic planning and safety rules provide predictable constraints. This is particularly appropriate for safety-critical autonomous driving, where an AI model can make a classification or confidence error.

The hybrid approach also improves explainability: the team can show a judge exactly why a vehicle slowed or stopped (for example, object class + estimated distance + safety threshold) rather than presenting an unexplained end-to-end neural-network decision.

22. 21. Judge Questions and Recommended Answers

Q: Why MATLAB instead of Python/CARLA/ROS?

A: MATLAB provides an integrated environment for scenario generation, vehicle modeling, control, visualization and deep-learning experimentation. This reduced integration overhead and allowed rapid testing of five targeted Indian-road cases.

Q: How do you handle roads without lane markings?

A: The system does not depend exclusively on lane lines. It uses free-space/waypoint-based navigation and is designed to use semantic road segmentation to estimate drivable regions and boundaries.

Q: What if YOLO misses a pedestrian or cattle?

A: A deterministic safety layer remains active. The design uses learned perception as an enhancement while retaining a geometric safety fallback capable of commanding braking when a hazard enters the protected region.

Q: Why is ReLU used?

A: ReLU introduces non-linearity while remaining computationally simple. It supports effective gradient flow for positive activations and is well suited to the convolutional feature-extraction pipeline.

Q: How do you prove the system works?

A: The master scenario runner executes all five environments and records objective metrics such as completion, minimum clearance, braking events, speed and latency. Visual logs and a performance dashboard provide additional evidence.

Q: Are the models actually trained?

A: The team should answer this precisely from the latest training logs: identify the dataset version, number of training/validation samples, model version, training completion status, and held-out evaluation metrics. Do not describe a planned or fallback model as fully trained unless the final artifact and evaluation results confirm it.

23. 22. Current Development Status

The documented work establishes a functioning MATLAB scenario framework and a five-scenario validation architecture. It also defines the deep-learning pipeline, ReLU-based CNN architecture, dataset parsing approach, safety fallback, pacing mechanism, visualization, metrics, and judge-defense material.

The main remaining technical task is to verify the final trained-model artifacts and integrate real inference into every scenario without degrading safety or stability. The team should also freeze a final dataset split, record reproducible training settings, benchmark each model separately, and regenerate the final five-scenario results from the exact code

version used for submission.

24. 23. Recommended Finalization Checklist

■ Verify all required MATLAB toolboxes and exact MATLAB release.

■ Freeze project folder/version and remove experimental duplicate scripts.

■ Confirm the final training/validation/test split.

■ Confirm YOLO model file and report mAP/precision/recall on held-out data.

■ Confirm DeepLabv3+ model and report IoU/Dice or equivalent segmentation metrics.

■ Confirm CNN behavioral-cloning model and report steering/speed MAE.

■ Confirm ReLU architecture in the final saved network.

■ Run all five scenarios from a clean MATLAB session.

■ Record completion, collisions, minimum clearance, braking events and latency.

■ Verify the reported results before putting them in the SIH PPT.

■ Record a clean demonstration video.

■ Prepare one architecture diagram and one data-flow diagram.

■ Prepare a 60–90 second explanation of the hybrid safety architecture.

■ Prepare answers to dataset, training, latency, failure-mode and scalability questions.

25. 24. Team Handover Summary

The project should be understood as a closed-loop autonomous-navigation system rather than five unrelated animations. The five scenarios are controlled experiments that expose different weaknesses of conventional lane-centric planning. Perception identifies road users and drivable space; prediction estimates short-term motion; decision logic chooses a behavioral response; planning/control generates a feasible movement; the vehicle executes it; and the safety monitor can override unsafe commands.

For team collaboration, each member should be able to explain one complete path through the architecture and also specialize in one layer: dataset/model training, perception, planning/control, scenario engineering, metrics/visualization, or SIH presentation. The most important discipline for the final submission is reproducibility: every reported model result and scenario metric should be traceable to a saved dataset split, model artifact, code version and test run.

Source and Status Note

This document consolidates the work recorded in the supplied project-development transcript. Where the record reports test results, those values are labeled as reported development-run results. They should be regenerated from the final frozen code and datasets before being presented as official SIH benchmarks.